

OOP

NTP II - Hafta 3



Bu hafta, Nesne Tabanlı Programlama (OOP) paradigmasının temel yapı taşlarını sağlam bir zemine oturtacağız.

Sınıf (Class) ve Nesne (Object) arasındaki kritik farkları anlamak.

Kurucu metotlar (Constructor/init), nitelikler (Attributes) ve metotların (Methods) işlevlerini ve tanımlanmasını öğrenmek.

Basit bir 'Ogrenci' sınıfı üzerinden pratik uygulama yapmak.



Nesne Tabanlı Programlama (OOP) Nedir?

OOP, yazılım tasarımı gerçek dünyadaki nesneler etrafında organize eden bir programlama paradigmasıdır.

Veri (nitelikler) ve bu veriler üzerinde çalışan işlevleri (metotlar) tek bir yapı içinde birleştirir.

Bu yaklaşım, büyük ve karmaşık sistemlerin yönetilebilirliğini ve esnekliğini artırır.

Simula (1960'lar) ilk nesne tabanlı dil olarak kabul edilir, ancak popülaritesi Smalltalk, C++ ve Java ile zirveye ulaşmıştır.



Neden OOP'yi Tekrar Ediyoruz?

OOP, modern yazılım mühendisliğinin temel taşıdır. İlerleyen konularda (Miras alma, Polimorfizm, Soyutlama) başarılı olmak, Class ve Object kavramlarını mükemmel anlamaya bağlıdır. Sınıf ve nesne arasındaki ilişki, tüm OOP prensiplerinin ana kaynağıdır. Bu hafta, soyut kavramları somut, endüstriyel örneklerle pekiştireceğiz.



Sınıf (Class) Kavramına Giriş

Sınıf, nesnelerin yapısını ve davranışını tanımlayan bir şablondur (blueprint) veya kalıptır.

Gerçek dünyadaki bir varlığın (örneğin Araba, Öğrenci, Hesap) soyut tanımını içerir.

Sınıf, bellekte herhangi bir yer kaplamaz; sadece nesnelerin nasıl oluşturulacağına dair kuralları barındırır.

Bir sınıftan birden çok nesne türetilabilir, bu nesnelerin hepsi aynı yapıya sahiptir ama farklı değerler taşıyabilir.

Sınıf Anatomisi: Nelerden Oluşur?

Bir sınıfın iki temel bileşeni vardır:

1. Nitelikler (Attributes/Fields/Properties): Nesnelerin sahip olduğu veriyi veya durumu tanımlar. (Örn: Bir 'Araba' sınıfı için renk, model, hız.)
 2. Metotlar (Methods/Functions): Nesnelerin yapabileceği eylemleri veya davranışları tanımlar. (Örn: Bir 'Araba' sınıfı için hızlan(), yavaşla(), dur().)
- Bu ikisi, bir araya gelerek veri ve işlevin bütünlüğünü (Encapsulation) sağlar.

Nesne (Object) Kavramı

Nesne, bir sınıfın somutlaştırılmış (instantiated) örneğidir.

Bir sınıftan oluşturulan, bellekte fiziksel olarak yer kaplayan ve kendine ait nitelik değerlerine sahip olan varlıktır.

Sınıf, 'Araba' fikriyse, nesne yolda gördüğünüz kırmızı bir Ford Fiesta'dır.

Her nesne, ait olduğu sınıfın metotlarını ve niteliklerini kullanabilir, ancak kendi durumunu (state) korur.



Sınıf ve Nesne: Araba Kalıbı Örneđi

Sınıfı, bir otomobil üretimindeki kalıp veya taslak olarak düşünün. Bu kalıp, otomobilin motorunun nerede olacağını, kaç tekerleđi olacağını ve hangi işlevlere sahip olacağını tanımlar. Nesneler ise bu kalıp kullanılarak üretim hattından çıkan somut arabalardır. Kalıp (Sınıf) tek ve sabittir. Üretilen arabalar (Nesneler) ise binlerce olabilir ve her birinin rengi, plakası, mevcut hızı farklıdır.



Nesne Durumu (State) ve Davranışı (Behavior)

Nesnenin Durumu (State), niteliklerinin mevcut değerleri ile tanımlanır.

Örnek: Bir 'Telefon' nesnesinin durumu: `batarya_seviyesi = 75`, `zil_sesi = 'titreşim'`.

Nesnenin Davranışı (Behavior), metotları aracılığıyla tanımlanır.

Örnek: Telefon'un davranışları: `arama_yap()`, `mesaj_gonder()`, `ses_ac()`.
Metotlar, genellikle nesnenin durumunu okur veya değiştirir.



Bellek Perspektifinden Sınıf ve Nesne

Sınıf tanımı, programın çalışması sırasında genellikle kod segmentinde (Code Segment) depolanır.

Sınıfın kendisi çalışma zamanında bellek tüketmez, sadece bir veri yapısının tarifidir.

Nesneler ise heap (yığın) alanında oluşturulur ve her nesne, kendi niteliklerinin değerlerini depolamak için bellek kaplar.

Bir programda 1000 'Ogrenci' nesnesi varsa, 1000 farklı bellek bloğu tahsis edilmiş demektir, ancak sınıf tanımı tektir.

Nesne Oluşturma Süreci (Instantiation)

Nesne oluşturma, sınıftan somut bir örnek (instance) yaratma işlemidir. Bu süreç, programlama diline göre farklılık gösterse de, temelde üç aşamadan oluşur:

1. Bellek Tahsisi: Nesnenin niteliklerini depolamak için yeterli bellek ayrılır.
2. Başlatma (Initialization): Kurucu metot (Constructor) çağrılarak nesnenin başlangıç durumuna getirilmesi sağlanır.
3. Referansın Döndürülmesi: Nesneye erişmek için kullanılan bir referans (işaretçi) döndürülür.



Kurucu Metot (Constructor) Nedir?

Kurucu metot, bir nesne yaratıldığı anda otomatik olarak çağrılan özel bir metottur.

Temel amacı, nesnenin niteliklerini (attributes) ilk (başlangıç) değerleriyle ayarlamaktır.

Python'da bu metot '`__init__`' olarak adlandırılır.

C++ veya Java gibi dillerde ise sınıf adıyla aynı ada sahiptir.

Kurucu metot olmazsa, nesne başlatılmaz ve nitelikler tanımsız veya varsayılan değerlerde kalır.



Kurucu Metodun Önemi

Kurucular, nesne oluşturulurken gerekli tüm koşulların yerine getirilmesini sağlar.

Zorunlu niteliklerin (örneğin bir öğrenci nesnesi için numara) eksiksiz girilmesini zorunlu kılar.

Nesnenin geçerli olmayan bir durumda başlatılmasını engeller, bu da programın güvenilirliğini artırır.

Kompleks nesneler için kaynakları (dosya bağlantıları, veritabanı oturumları) hazırlamak için de kullanılabilirler.

Python'da Kurucu Metot: `__init__`

Python'da kurucu metod, özel bir dunder (double underscore) metodu olan `__init__` olarak tanımlanır.

Tanım: `def __init__(self, parametre1, parametre2, ...):`

'self', oluşturulan nesnenin kendisine bir referanstır ve her metotta ilk parametre olmak zorundadır.

Parametreler, nesne oluşturulurken dışarıdan alacağımız başlangıç değerleridir.

Kurucu metotlar, açıkça bir değer döndürmezler (otomatik olarak oluşturulan nesne referansını döndürürler).

'self' Anahtar Kelimesinin Rolü

'self' (veya Java/C++'daki 'this'), metot içinde o an üzerinde çalışılan nesnenin kendisini temsil eder.

Nesne niteliklerine erişmek ve onları ayarlamak için hayati öneme sahiptir.

Örnek: 'self.ad = disaridan_gelen_ad' ifadesinde, 'self.ad' nesnenin niteliğini, 'disaridan_gelen_ad' ise kurucuya dışarıdan gelen değeri ifade eder.

'self' olmadan, yerel değişkenler ile nesne niteliklerini karıştırma riski doğar.

Varsayılan Kurucular

Eğer bir sınıf içinde açıkça bir kurucu metod tanımlamazsak, birçok dil otomatik olarak parametresiz bir varsayılan kurucu sağlar. Bu kurucu, nesneyi başlatır ancak niteliklere özel başlangıç değerleri atamaz; genellikle sıfır veya null gibi varsayılan değerler kullanılır. Eğer bir sınıf parametrelili bir kurucuya sahipse, varsayılan kurucu otomatik olarak sağlanmayabilir (Python'da bu durum esnektir, ancak Java/C++'da dikkat gerektirir).



Parametrelili Kurucular

Parametrelili kurucular, nesne oluşturulurken belirli verilerin sağlanmasını zorunlu kılar.

Örnek: Bir 'KrediHesabi' nesnesi oluşturulurken mutlaka 'hesap_sahibi' ve 'ilk_yatirim_miktari' gibi değerlerin girilmesi gerekebilir.

Bu, nesnenin eksik veya geçersiz veriyle çalışmaya başlamasını önler. Kurucunun aldığı parametreler, nesnenin ilk durumunu tam olarak tanımlamalıdır.



Kurucu Metotlarda Aşırı Yükleme (Overloading)

Bazı dillerde (Java, C++), bir sınıfa birden fazla kurucu metot tanımlanabilir, buna Aşırı Yükleme (Overloading) denir.

Her kurucu, farklı sayıda veya türde parametre alabilir.

Örnek: Bir 'Ogrenci' sınıfı hem 'ad' ve 'numara' ile, hem de sadece 'numara' ile başlatılabilir.

Python, teknik olarak kurucu aşırı yüklemesini doğrudan desteklemez; bunun yerine, isteğe bağlı parametreler (default values) kullanarak tek bir '__init__' metoduyla farklı başlatma senaryolarını yönetmeyi tercih eder.

Nitelikler (Attributes) Tanımı

Nitelikler, bir sınıfın veya nesnenin verilerini (durumunu) temsil eden değişkenlerdir.

OOP'de nitelikler, ilgili metotlarla (davranışlarla) birlikte kapsülленir. İki temel nitelik türü vardır: Sınıf Nitelikleri ve Nesne (Örnek) Nitelikleri.

Nesne Nitelikleri (Instance Attributes)

Nesne nitelikleri, bir sınıftan oluşturulan her bir nesneye özgü olan verilerdir. Bu nitelikler genellikle kurucu metot ('__init__') içinde tanımlanır ve 'self' anahtar kelimesi ile ilişkilendirilir. Bir nesnenin niteliğinde yapılan değişiklik, diğer nesneleri etkilemez. 'Ogrenci' örneğinde, 'ad' ve 'numara' nesne nitelikleridir.

Sınıf Nitelikleri (Class Attributes)

Sınıf nitelikleri, sınıf tanımının hemen altında, metotların dışında tanımlanır. Bu nitelikler, o sınıftan oluşturulan tüm nesneler tarafından ortak olarak paylaşılır.

Genellikle sabitleri (constants) veya sayım sayaçlarını (counter) tutmak için kullanılır.

Örnek: 'Araba' sınıfında, tüm arabaların kullandığı 'TekerlekSayisi = 4' bir sınıf niteliği olabilir.

Sınıf ve Nesne Nitelikleri Arasındaki Kritik Fark

Nesne nitelikleri: Bellekte her nesne için ayrı bir kopya tutulur. Değişiklik sadece o nesneyi etkiler.

Sınıf nitelikleri: Bellekte tek bir kopya tutulur. Bir nesne üzerinden bu nitelik değiştirilirse, bu değişiklik aynı sınıftan türetilmiş tüm nesneler için geçerli olur (Eğer değişiklik sınıf düzeyinde yapılırsa).

Üniversite düzeyinde bilinmesi gereken: Nesneler, sınıf niteliklerini okuyabilir, ancak kendi üzerlerinde yeniden tanımlarlarsa, bu bir nesne niteliği haline gelir (gölgeleme).



Metotlar (Methods) Tanımı

Metotlar, bir sınıf içinde tanımlanan işlevlerdir ve nesnelerin belirli eylemleri gerçekleştirmesini sağlar.

Metotlar, genellikle nesnenin niteliklerini okur, değiştirir veya bunlar üzerinde hesaplamalar yapar.

Tüm örnek metotlarının ilk parametresi, nesnenin kendisine referans veren 'self' (veya 'this') olmak zorundadır.

Metotlar, karmaşık işlemleri daha küçük, yönetilebilir birimlere ayırarak kodun okunabilirliğini artırır.



Metot Türleri: Örnek Metotları (Instance Methods)

En yaygın kullanılan metot türüdür.

Bu metotlar, belirli bir nesne örneği üzerinde çalışır ve o nesnenin niteliklerine erişebilir ve bunları değiştirebilir.

'self' parametresini alırlar ve nesnenin durumuna (state) doğrudan bağlıdırlar.

Örnek: Bir 'BankaHesabi' nesnesindeki 'para_yatir()' metodu, yalnızca o hesabın bakiyesini değiştirir.

Metot Türleri: Sınıf Metotları (Class Methods)

Sınıf metotları, belirli bir nesne örneğine değil, sınıfın kendisine bağlıdır. Bunlar, genellikle sınıf nitelikleri üzerinde işlem yapmak veya sınıf için alternatif kurucular oluşturmak için kullanılır. Python'da '@classmethod' dekoratörü ile tanımlanır ve ilk parametre olarak 'cls' (sınıfın kendisine referans) alır. Nesne durumu yerine sınıf durumunu (tüm nesneler için ortak veriyi) yönetirler.

Metot Türleri: Statik Metotlar (Static Methods)

Statik metotlar, ne nesneye ('self') ne de sınıfa ('cls') bağlıdır. Bunlar, sınıf tanımı içinde mantıksal olarak yer alması gereken ancak ne nesnenin durumuna ne de sınıfın durumuna ihtiyaç duyan yardımcı işlevlerdir. Python'da '@staticmethod' dekoratörü ile tanımlanır. Örnek: Bir 'Tarih' sınıfı içinde 'tarih_formatini_kontrol_et()' gibi bir yardımcı işlev.

Basit 'Ogrenci' Sınıfı Uygulamasına Giriş

Kaynakta belirtildiği gibi, temel bir 'Ogrenci' sınıfı oluşturarak Class, Object, Attribute ve Method kavramlarını birleştirelim.

İlk adım, 'class' anahtar kelimesi ile sınıfı tanımlamaktır.

Örnek Yapı:

```
'class Ogrenci:'  
' # Sınıf nitelikleri buraya gelir (varsa)'  
' def __init__(...):'  
'     # Nesne nitelikleri başlatılır'  
' def metot_adi(...):'  
'     # Davranış tanımlanır'
```

'Öğrenci' Sınıfının Nitelikleri

Öğrenci nesnesinin temel nitelikleri şunlar olacaktır:

1. 'ad' (String): Öğrencinin adı.

2. 'numara' (Integer/String): Öğrencinin okul numarası veya kimliği.

Bu nitelikler, nesne oluşturulduğunda zorunlu olarak verilmelidir, bu nedenle '__init__' metodu içinde tanımlanacaklardır.

'Ogrenci' Sınıfı: Kurucu Metot (__init__) Tanımı

Ogrenci sınıfının kurucusunu, 'ad' ve 'numara' parametrelerini alacak şekilde tasarlayalım:

```
'def __init__(self, ad, numara):'
```

Bu parametreleri kullanarak, 'self' üzerinden nesne niteliklerini atarız:

```
'self.ad = ad'
```

```
'self.numara = numara'
```

Bu, her yeni Ogrenci nesnesinin, kendi ad ve numara değerlerini taşımasını sağlar.

'Ogrenci' Sınıfı: 'bilgileri_goster()' Metodu

Kaynakta belirtilen 'bilgileri_goster()' metodu, nesnenin mevcut durumunu (ad ve numara) ekrana yazdırma davranışını temsil eder.

Bu bir örnek metodu olduğu için 'self' parametresini almalıdır:

```
'def bilgileri_goster(self):'
```

Metot içeriği, 'self' kullanarak nesnenin niteliklerine erişir:

```
'print(f'Öğrenci Adı: {self.ad}')
```

```
'print(f'Öğrenci Numarası: {self.numara}')
```

Nesne Oluşturma ve Metot Çağırma

Sınıf tanımını yaptıktan sonra, ondan nesneler (öğrenciler) üretebiliriz:

```
'ogr1 = Ogrenci('Ayşe Yılmaz', 2023001)'
```

```
'ogr2 = Ogrenci('Mehmet Kaya', 2023002)'
```

Nesneler üretildikten sonra, onların davranışlarını çağırabiliriz:

```
'ogr1.bilgileri_goster()'
```

Her çağrı, o nesneye ait benzersiz veriyi (Ayşe Yılmaz) kullanır.

OOP Prensiplerine Geçiş: Kapsülleme (Encapsulation)

Sınıf ve nesne kavramı, temel olarak Kapsüllemeyi (Encapsulation) uygular. Kapsülleme, veriyi (nitelikleri) bu veri üzerinde işlem yapan metotlarla bir araya getirme ve dış dünyadan saklama prensibidir.

Bu, veriye doğrudan erişimi kısıtlayarak (genellikle özel/private nitelikler kullanarak) veri bütünlüğünü korur.

'Ogrenci' örneğinde, ad ve numara verileri 'Ogrenci' sınıfının içine kapsüllenmiştir.



Niteliklere Güvenli Erişim: Getters ve Setters

Akademik ve kurumsal projelerde, niteliklere doğrudan erişmek yerine metotlar (Getter/Setter) aracılığıyla erişmek yaygındır.

Getter Metotları: Niteliklerin değerini okumak için kullanılır (Örn: 'get_ad()').

Setter Metotları: Niteliklerin değerini değiştirmek için kullanılır ve bu esnada doğrulama (validation) yapılmasına olanak tanır (Örn: 'set_numara(yeni_numara)').

Bu, verinin geçerli sınırlar içinde kalmasını sağlar.



Python'da Veri Gizleme Mekanizması

Python, Java veya C++'daki gibi katı 'private' (özel) erişim belirteçlerine sahip değildir.

Bunun yerine, bir niteliğin adının başına çift alt çizgi ('__') koyarak isim karıştırması (name mangling) yapılır.

Örn: 'self.__gizli_sifre'

Bu, niteliğe dışarıdan doğrudan erişimi zorlaştırır, ancak tamamen engellemez. Bu yapı, geliştiriciye 'dokunma' uyarısıdır.

Property Dekorasyonu (Academic View)

Üniversite düzeyinde, Python'da 'property' dekoratörünün bilinmesi önemlidir. 'property' dekoratörü, metotları nitelik gibi kullanmamızı sağlar. Böylece dışarıdan erişim bir nitelik gibi görünse de, arka planda veri doğrulama mantığı çalıştırılabilir. Bu, hem kapsüllemeyi sağlar hem de kodun temiz (clean) kalmasına yardımcı olur.



Sınıf Yaşam Döngüsü: Yaratma ve Yok Etme

Nesne Yaratma: Kurucu metot ile başlar.

Nesne Kullanımı: Metotlar ve nitelikler aracılığıyla etkileşim kurulur.

Nesne Yok Etme: Nesneye olan tüm referanslar kaybolduğunda, programlama dilinin çöp toplayıcısı (Garbage Collector) devreye girer.

Python'da '`__del__`' metodu (destructor), nesne bellekten silinmeden hemen önce çağrılır. Bu genellikle kaynakları serbest bırakmak için kullanılır (dosyaları kapatmak gibi).



Dunder Metotlar (Magic Methods) I

Dunder metotlar, iki alt çizgi ile başlayıp biten özel metotlardır (örnek: `'__init__'`, `'__str__'`).

Bu metotlar, Python'un yerleşik işlevlerinin (operatörlerin, fonksiyonların) sınıflarımızla nasıl etkileşim kuracağını tanımlar (Operator Overloading). Bu sayede kendi tanımladığımız nesneler, listeler, diziler veya matematiksel operatörler gibi standart dillerle doğal bir şekilde çalışabilir.



Dunder Metotlar II: `__str__` ve `__repr__`

'`__str__`'(self)': Nesnenin kullanıcılar tarafından okunabilir, resmi olmayan dize temsilini döndürür. Genellikle 'print()' fonksiyonu tarafından çağrılır.

'`__repr__`'(self)': Nesnenin geliştiriciler tarafından okunabilir, resmi dize temsilini döndürür. Nesneyi yeniden oluşturmaya yetecek bilgiyi içermelidir. Bu metotların tanımlanması, nesnelerin hata ayıklanmasını (debugging) ve loglanmasını büyük ölçüde kolaylaştırır.

Operator Overloading (Operatör Aşırı Yükleme)

Operatör aşırı yükleme, standart operatörlerin (+, -, *, < vb.) kendi sınıflarımız için farklı anlamlar taşımasını sağlamaktır.

Örn: İki 'Vektör' nesnesini toplamak istediğimizde, '+' operatörünü '__add__' dunder metodu ile yeniden tanımlayarak vektörel toplama yaptırabiliriz. Bu, kodu daha sezgisel ve matematiksel olarak doğru hale getirir.



Karşılaştırma Operatörlerinin Aşırı Yüklenmesi

Nesnelerin eşitliğini kontrol etmek için '==eq==' (eşit), '==ne==' (eşit değil), '==lt==' (küçük) gibi dunder metotları kullanılır.

Varsayılan olarak, iki nesne yalnızca bellekte aynı referansa sahipse eşittir.

'==eq==' metodu, bizim belirlediğimiz niteliklere göre (Örn: iki öğrencinin numaraları aynıysa eşittir) eşitliği tanımlamamızı sağlar.

Bu, veritabanı sorgularında veya set yapılarında nesne yönetimini kritik hale getirir.



Nesne Etkileşimi: Argüman Olarak Nesne Kullanımı

OOP'de, bir nesneyi başka bir nesnenin metotlarına argüman olarak geçirmek çok yaygındır.

Örnek: Bir 'Sınav' sınıfının 'sinav_yap()' metodu, argüman olarak bir 'Ogrenci' nesnesi alabilir.

Bu, farklı sınıfların birbirlerinin davranışları üzerinde işlem yapmasını ve karmaşık sistemlerin etkileşimini mümkün kılar.

Bu geçiş genellikle referansla (pass by reference) yapılır, yani metot nesnenin durumunu değiştirebilir.



Soyutlama (Abstraction) Kavramı

Soyutlama, kullanıcının yalnızca bir nesnenin temel işlevlerine odaklanması, karmaşık iç detayların gizlenmesi prensibidir.

Kullanıcı, araba sürerken motorun iç işleyişini bilmek zorunda değildir; sadece gaza basma eylemini bilir.

Sınıflar, arayüzler ve soyut metotlar (Abstract Methods) soyutlamanın temel araçlarıdır.

OOP, sınıf ve nesne tanımıyla doğal olarak soyutlamayı teşvik eder.

Arayüz (Interface) ve Sözleşme Kavramı

Arayüz, bir sınıfın sahip olması gereken metotları tanımlayan bir sözleşmedir. Bir sınıf bu arayüzü uyguladığında, bu metotları mutlaka kendi içinde tanımlamak zorundadır.

Python'da doğrudan 'interface' yapısı yoktur; bunun yerine Soyut Temel Sınıflar (Abstract Base Classes - ABC) modülü kullanılır.

Bu, farklı sınıfların aynı davranışı sergilemesini (polimorfizme zemin hazırlar) sağlar.



Basit Sınıf Yapısını Genişletme: 'Ders' Nitelikleri

Daha karmaşık bir 'Ogrenci' sınıfı oluştururken, sadece temel nitelikler yetersiz kalır.

Öğrencinin aldığı dersleri veya notları bir liste/sözlük (list/dictionary) olarak tutabiliriz.

Örnek nitelik: 'self.dersler = {}' (Ders adı: Not şeklinde).

Kurucuda başlatılan bu veri yapıları, daha sonra metotlar (Örn: 'ders_ekle()', 'not_gir()') aracılığıyla manipüle edilir.

Kodlama Standartları ve İsimlendirme Kuralları

Üniversite düzeyinde kod yazarken standartlara uymak önemlidir (Python için PEP 8).

Sınıf isimleri: PascalCase (Örn: Öğrenci, BankaHesabi).

Metot ve Nitelik isimleri: snake_case (Örn: bilgileri_goster, ogrenci_numarasi).

Bu standartlara uymak, kodun endüstriyel olarak okunabilir ve sürdürülebilir olmasını sağlar.



İstisnai Durum Yönetimi (Exception Handling) ve Sınıflar

Sağlam bir OOP tasarımı, geçersiz veri girişini engellemelidir.

Örn: Eğer bir 'Öğrenci' nesnesi oluşturulurken 'numara' negatif girilirse, kurucu metot içinde bir istisna (Exception) fırlatılmalıdır.

```
'if numara <= 0: raise ValueError('Öğrenci numarası pozitif olmalıdır.')
```

Bu yaklaşım, nesnenin geçersiz bir durumda oluşturulmasını kökten engeller.

Fabrika Metotları (Factory Methods) Kavramı

Fabrika metotları, nesne oluşturma mantığını bir metot içine kapsülleyen tasarım desendir.

Bu, kullanıcıyı '.__init__' metodunun karmaşık parametrelerinden kurtarır.

Örnek: 'Ogrenci.kayittan_olustur(kayit_dosyasi)' statik metodu, veriyi bir dosyadan okuyup yeni bir Ogrenci nesnesi döndürebilir.

Bu, nesne oluşturma sürecine esneklik ve kontrol katar.



Sınıfların Modülerliği ve Paket Yapısı

Gerçek dünya yazılımlarında, her sınıf tek bir dosyada (modülde) bulunur. 'Ogrenci' sınıfı 'ogrenci.py' modülünde yer almalıdır. İlgili modüller bir araya gelerek paketleri (packages) oluşturur. Bu modüler yapı, kodun yeniden kullanılabilirliğini (reusability) ve ekip çalışmasına uygunluğunu artırır.



Nesne İlişkileri I: 'Has-A' ilişkisi (Composition)

Kompozisyon (Composition), bir sınıfın başka bir sınıftan nesneleri nitelik olarak içermesidir.

Bu, 'sahip olma' ('has-a') ilişkisini tanımlar.

Örnek: 'Öğrenci' sınıfı, bir 'Adres' sınıfının nesnesini nitelik olarak içerebilir (Bir öğrencinin bir adresi vardır).

Bu, karmaşık nesneleri daha küçük ve yönetilebilir parçalara ayırarak sistem tasarımını basitleştirir.



Nesne İlişkileri II: Toplama (Aggregation)

Toplama (Aggregation) da bir 'has-a' ilişkisidir, ancak kompozisyona göre daha gevşektir.

İçerilen nesne, ana nesne olmadan da varlığını sürdürebilir.

Örnek: Bir 'Sınıf' nesnesi, bir 'Öğrenci' nesneleri listesini içerebilir. Sınıf dağıtılsa bile, öğrenciler var olmaya devam eder.

Bu ilişki, bağımsız nesnelerin bir araya gelerek daha büyük bir sistemi oluşturduğunu gösterir.



Python, referans sayımı denen bir sistem kullanarak çöp toplama (Garbage Collection) yapar.

Bir nesneye yapılan her referans, sayacı 1 artırır.

Bu sayaç sıfıra düştüğünde (yani nesneye işaret eden kimse kalmadığında), nesne otomatik olarak bellekten silinir.

Bu mekanizma, programcının manuel bellek yönetim yükünü azaltır, ancak referans döngüleri (circular references) problemine yol açabilir.



Kullanıcı Tanımlı Veri Tipleri Olarak Sınıflar

OOP'de sınıflar, basit türler (int, string) gibi, programcı tarafından tanımlanmış yeni veri tipleri oluşturur.

'Ogrenci' artık bir türdür.

Modern Python'da Type Hinting (Tür ipucu) kullanarak kodun okunabilirliğini ve statik analiz araçlarının (linters) hata bulma yeteneğini artırabiliriz.

Örn: `def kaydet(ogrenci: Ogrenci):`

Tasarım Desenleri ve Sınıf Kavramı

Sınıf ve nesne kavramları, yazılım tasarım desenlerinin temelini oluşturur.

Singleton Deseni: Bir sınıftan sadece tek bir nesnenin oluşturulmasını garanti eder (Örn: Uygulama ayarları sınıfı).

Observer Deseni: Nesneler arasında bire-çok bağımlılık kurarak bir nesne değiştiğinde tüm bağımlıların otomatik güncellenmesini sağlar.

Bu desenler, sınıf tasarımının karmaşık problemleri çözmek için nasıl kullanıldığını gösterir.



Örnek Uygulama: Sınıf Niteliklerinin Kullanımı

Ogrenci sınıfına, kaç tane öğrenci nesnesi oluşturulduğunu sayan bir sınıf niteliği ekleyelim:

```
'class Ogrenci:'
```

```
'    ogrenci_sayisi = 0'
```

Bu sayacı, her '__init__' çağrıldığında artırmalıyız:

```
'def __init__(self, ad, numara):'
```

```
'    Ogrenci.ogrenci_sayisi += 1'
```

Bu sayaç, tüm nesneler tarafından ortaklaşa kullanılır.

Kodlama Örneği: 'Ogrenci' Sınıfı (Kurucu ve Nitelikler)

```
class Ogrenci:
    ogrenci_sayisi = 0

    def __init__(self, ad, numara):
        self.ad = ad
        self.numara = numara
        Ogrenci.ogrenci_sayisi += 1

    def bilgileri_goster(self):
        return f'Ad: {self.ad}, Numara: {self.numara}'

# Nesne Oluşturma
ogr_a = Ogrenci('Ali', 101)
print(ogr_a.bilgileri_goster())
```

Kodlama Örneği: Nesneler Arası Etkileşim

```
ogr_b = Ogrenci('Deniz', 102)
# Niteliklere Doğrudan Erişim (Pythonic)
print(f'{ogr_b.ad} öğrencisinin numarasını değiştiriyorum...')
ogr_b.numara = 103
# Sınıf niteliğine erişim
print(f'Toplam Öğrenci Sayısı: {Ogrenci.ogrenci_sayisi}')
# Not: Sınıf sayacına hem sınıf hem de nesne üzerinden erişilebilir, ancak sınıf
üzerinden değiştirmek daha doğrudur.
```


Nesne Tabanlı Düşünme: Gerçek Hayat Problem Çözümü

Bir kütüphane yönetim sistemi tasarlarken:

1. Nesneleri Belirleme: Kitap, Üye, Kütüphane, İşlem.
 2. Sınıf Tanımlama: Her nesne için bir sınıf (Örn: 'Kitap' sınıfı).
 3. Nitelik Ekleme: Kitap'a 'ISBN', 'Yazar', 'Durum' nitelikleri.
 4. Davranış Ekleme: Kitap'a 'odunc_ver()', 'iade_al()' metotları.
- Bu ayrıştırma, kodun bakımını ve ölçeklenmesini kolaylaştırır.

İmmutabilite (Değişmezlik) ve Nesneler

İmmutabilite, bir nesnenin oluşturulduktan sonra durumunun (niteliklerinin değerlerinin) değiştirilememesi anlamına gelir.

Değişmez nesneler, özellikle çoklu iş parçacıklı (multi-threaded) ortamlarda ve fonksiyonel programlama tarzlarında güvenliği artırır.

Bazı diller immutabiliteyi zorunlu kılarken, Python'da genellikle nesne niteliklerini private yaparak ve setter metotlarını iptal ederek sağlanır.

Sınıf ve Nesne: Soyutlama Düzeyleri

Sınıflar, programın Yüksek Düzeyli Tasarımını (High-Level Design) temsil eder (Ne yapılacağını tanımlar).

Nesneler, programın Çalışma Zamanı (Runtime) Gerçekleşmesini temsil eder (Nasıl yapılacağını ve hangi verilerle çalışılacağını tanımlar).

Bu ayırım, yazılım mimarisini katmanlara ayırmak ve bağımlılıkları yönetmek için temeldir.



Kapanış ve Özet: Sınıf Nedir?

Sınıf, bir veri yapısı ve davranışlarının mantıksal tanımını (şablonunu) içeren soyut bir veri tipidir.

Bellekte yer kaplamaz, nesnelerin nasıl inşa edileceğini belirtir.

Nitelikleri (veri) ve metotları (davranış) kapsüller.



Kapanış ve Özet: Nesne Nedir?

Nesne, bir sınıfın somut, çalışan bir örneğidir (instance). Bellekte yer kaplar ve kendine ait, benzersiz nitelik değerlerini (durumu) barındırır. Sınıfın tanımladığı metotlar aracılığıyla dış dünya ile etkileşime girer.

Kapanış ve Özet: Kurucu Metotlar

Kurucu metot ('__init__'), nesne oluşturma anında çağrılan özel bir metottur. Amacı, yeni oluşturulan nesnenin niteliklerini geçerli ve zorunlu başlangıç değerleriyle ayarlamaktır. 'self' parametresini alarak, o anki nesne referansını kullanır.

Kapanış ve Özet: Nitelikler ve Metotlar

Nitelikler (Attributes): Nesnenin durumunu ve verilerini tutar (Örn: ad, numara).
Metotlar (Methods): Nesnenin davranışlarını tanımlar ve durumu değiştirir veya okur (Örn: bilgileri_goster()).
Bu ikilinin birleşimi, OOP'nin temel yapı taşıdır.



Dört Temel OOP Prensipleri (Miras, Polimorfizm, Kapsülleme, Soyutlama).
Tasarım Desenleri (Design Patterns).
Dependency Injection (Bağımlılık Enjeksiyonu).
Statik Tip Kontrolü (Type Checking) araçlarının (mypy) OOP ile kullanımı.

NTP II

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

